

Ingeniería y Calidad de Software

Notas de repaso para el primer parcial · 29/09/2026

1. De los temas a las User Stories

Los requerimientos pueden ubicarse en distintos niveles de detalle. Los **temas de inversión** son agrupadores amplios de necesidades o áreas del producto; las **épicas** expresan funcionalidades demasiado grandes para resolverse en una sola iteración; y las **User Stories** son unidades más pequeñas con las que el equipo puede comprometer trabajo en un Sprint. Durante la elicitación no hace falta adivinar de entrada a qué categoría pertenece cada necesidad: el tamaño se reconoce al analizarla.

Una historia que se creía pequeña puede resultar una épica cuando se descubre que no entra en el Sprint. Se la divide en historias que sí puedan completarse dentro de una iteración, pero cada parte debe conservar **valor para el usuario o el negocio**. Cortar por capas técnicas hasta obtener historias diminutas que no aportan valor no resuelve el problema.

Para entrar al Sprint, una User Story debe cumplir la **Definition of Ready** del equipo. Como base se emplea **INVEST**: independiente, negociable, valiosa, estimable, pequeña y comprobable. «Pequeña» es relativa a la duración y capacidad del Sprint: una historia que entra en uno de cuatro semanas quizá no entra en uno de dos. El equipo puede agregar condiciones a su Definition of Ready.

Product Backlog, Sprint Backlog y tipos de ítems

El **Product Backlog** es una lista ordenada de necesidades del producto. Sus elementos reciben el nombre genérico de *Product Backlog Items* (PBI). Puede contener temas, épicas, historias, spikes, defectos, deuda técnica y necesidades no funcionales, siempre que representen trabajo vinculado al producto. Las User Stories son una técnica frecuente, pero Scrum no obliga a usarlas; se podrían especificar requisitos con otros formatos. El Product Backlog **no es una lista de tareas técnicas**.

En el **Sprint Backlog** se selecciona trabajo concreto para el Sprint. Una épica o un tema no ingresan como unidad implementable porque exceden la iteración o carecen del detalle necesario. En el enfoque usado en clase entran User Stories listas, y también puede reservarse capacidad para resolver spikes. Si el equipo descompone historias en tareas técnicas, esas tareas aparecen en la planificación y en el tablero del Sprint.

Spikes e incertidumbre

Un **spike** representa una investigación acotada necesaria cuando la incertidumbre impide estimar o imaginar la solución de un requerimiento. Puede descubrirse al intentar estimar una historia o detectarse directamente. No importa tanto si «la historia se transforma en spike» o si «genera un spike»: importa identificar qué información falta y dedicar trabajo a conseguirla.

Tipo	Incertidumbre y responsabilidad principal
Funcional	Falta una definición del negocio o de las reglas que debe cumplir el producto. El Product Owner es el referente para resolver esa definición, con la colaboración necesaria.
Técnico	No está claro cómo implementar, integrar o elegir una tecnología. El equipo técnico investiga y prueba alternativas.

Una misma funcionalidad puede tener incertidumbre funcional y técnica. El refinamiento permite conversar y aclarar requerimientos, pero una investigación de ingeniería puede exigir análisis, diseño, pruebas y tiempo específico dentro de un Sprint. Por eso se **timeboxea** el spike: se asigna un período limitado de investigación y se vuelve con resultados para poder estimar o decidir. Estimar el spike significa estimar ese tiempo de investigación, no fingir que ya se conoce el esfuerzo de implementación. Si la incertidumbre persiste, se podrá dedicar otro período en un Sprint posterior.

No conviene incluir en el mismo Sprint el spike y la historia cuya implementación depende de que ese spike se resuelva. El Sprint también debe contener historias listas que permitan generar un incremento. Refinamiento, investigación, pruebas, Review y Retrospective consumen tiempo del Sprint; dejar trabajo técnico o testing

para una etapa externa de «endurecimiento» contradice la idea de producir un incremento terminado en cada iteración.

2. Validación del producto, MVP y releases

Cuando existe una idea de producto todavía incierta, se decide qué porción mínima conviene mostrar o liberar para **validar una hipótesis**: interés de usuarios, aceptación de una solución, apoyo de inversores, etc. La versión inicial no tiene por qué coincidir con la visión completa. El feedback puede llevar a modificarla y probar una nueva versión; también se puede sostener una decisión original si hay razones para hacerlo. Es un proceso iterativo de aprendizaje, sin una receta única.

Término	Qué se busca
MVF / MVP	Minimum Viable Feature / Product : una sola característica o un conjunto mínimo de características para validar una hipótesis sobre el producto. Lo central es aprender si vale la pena avanzar, no completar todo el producto.
MMF / MMP	Minimum Marketable Feature / Product : característica o versión mínima comercializable, suficientemente valiosa para ponerla en producción y ofrecerla al mercado. Aquí ya se está decidiendo qué versión mínima se lanzará.

La letra **F** indica una característica; la **P**, un conjunto que se trata como producto. La **V** se asocia con la validación de una hipótesis y la **M** con la versión mínima para comercializar o poner en producción. También puede validarse una hipótesis sobre un **producto que ya existe**, por ejemplo antes de agregar una funcionalidad importante.

Un **release** es una versión del producto que se libera. El producto evoluciona como una secuencia de releases: la segunda versión agrega o modifica lo disponible en la primera, y así sucesivamente. Un MVP puede ser el primer release si se libera para validar una hipótesis; cuando ya se decidió salir a producción, el primer release puede ser un MMP. «Release» destaca la versión entregada en un momento; «producto» destaca la entidad que continúa evolucionando.

El **Roadmap** proyecta a alto nivel qué características se espera ofrecer y aproximadamente cuándo, por ejemplo por trimestre o semestre. De allí se desprende la planificación de releases. Cada release se identifica como una versión para poder reconocer qué funcionalidades estaban disponibles en cada momento.

3. Roles, revisión del incremento y cambios en Scrum

Product Developer es el nuevo nombre empleado en la expansión de 2026 para quienes antes se llamaban *Developers*; no representa un rol completamente diferente. Puede tratarse de una persona, de una persona asistida por IA o de un agente automatizado. Product Owner y Scrum Master siguen siendo roles humanos.

Human in the loop significa que siempre debe haber supervisión humana del trabajo generado por IA. El equipo autogestionado define cómo realizará sus revisiones técnicas; no hace falta crear un jefe de las IA. La revisión de artefactos busca detectar errores de forma temprana y corresponde también cuando el trabajo fue producido por personas: cualquiera puede equivocarse. La responsabilidad es del equipo.

La **Sprint Review** no es solo una reunión privada para mostrarle algo al Product Owner. Participan el equipo y los interesados pertinentes para inspeccionar lo producido, recibir feedback y ajustar el rumbo. Se presentan incrementos e historias que cumplieron la **Definition of Done**; una historia incompleta no se exhibe como terminada. Si no hay nada terminado, existe un problema serio para inspeccionar valor, aunque el evento de Scrum no debería eliminarse por esa razón.

La duración del **Sprint** es fija y de **un mes o menos**. Sprint Planning, Daily, Review y Retrospective tienen duraciones máximas; por ejemplo, la Planning puede durar hasta ocho horas para un Sprint de un mes y la Daily hasta quince minutos. El **refinamiento** ocurre continuamente según la necesidad; se mencionó como referencia dedicarle alrededor del 10 % del tiempo del Sprint, no programarlo obligatoriamente en un único momento.

Regla para los cambios durante el Sprint: el criterio general es proteger el Sprint Goal. No se deberían aceptar nuevos requerimientos que lo pongan en riesgo ni sumar trabajo solo porque alguien lo pide. Se puede aclarar o renegociar el alcance con el Product Owner cuando el objetivo se mantiene y el equipo considera que dispone de capacidad. Agregar funcionalidad aun así implica riesgo; una solicitud ajena al objetivo debe quedar para otra planificación.

4. Producto, proyecto, ciclo de vida y proceso

El **ciclo de vida del producto** abarca su trayectoria completa: idea, introducción, madurez, posible declive y retiro. Puede durar mucho más que un proyecto. El **ciclo de vida del proyecto o del desarrollo** describe cómo se organiza el trabajo para obtener un release particular. Puede ser en cascada, iterativo e incremental, recursivo, etc. Por eso un producto puede atravesar varios proyectos y releases a lo largo de su existencia.

El **proceso** reúne las actividades necesarias para construir, entregar y gestionar software: requerimientos, diseño, construcción, pruebas, planificación, gestión de configuración, calidad y despliegue, entre otras. Al iniciar un proyecto se adapta el proceso a su contexto. El ciclo de vida elegido condiciona **cuántas veces, en qué orden y con qué artefactos** se ejecutan esas actividades. Con un enfoque iterativo habrá planes por iteración; con una cascada estricta se concentrará gran parte de la planificación al comienzo.

La administración del proyecto forma parte del proceso: planificar, monitorear y controlar el trabajo no es algo externo a la construcción. También influyen el tamaño y experiencia del equipo, la distribución geográfica, el trabajo presencial o remoto y otros rasgos concretos del proyecto.

5. Planificación adaptativa: de la estrategia al día

La planificación se hace a diferentes escalas porque el producto completo normalmente no se construye en un Sprint. Cuanto más lejano es el horizonte, menor detalle conviene fijar; las decisiones detalladas se posponen hasta contar con información suficiente. La inspección y adaptación pueden obligar a rehacer un plan cuando cambian las prioridades o la duración de los Sprints.

Nivel	Pregunta y alcance
Portafolio	Qué productos se lanzan, sostienen o retiran en una organización con varios productos.
Visión, Roadmap y plan de producto	Qué producto se quiere y qué características se proyectan a alto nivel para próximos períodos. El Roadmap orienta, no promete detalle inmutable.
Plan de release	Qué conjunto de características se espera entregar en una versión y cuándo. Abarca varios Sprints; se desarrolla en detalle el release más cercano y se ajusta con el aprendizaje.
Sprint Planning	Cuál será el Sprint Goal, qué historias se seleccionan según la capacidad y cómo se trabajará. Produce el Sprint Backlog y fija condiciones como duración y capacidad.
Daily Scrum	Qué hacer y cómo adaptar el trabajo inmediato para avanzar hacia el Sprint Goal.

El **release pertenece al producto**; un proyecto puede tener como resultado un release. Roadmap y planes de producto y release miran un horizonte mayor, mientras que la Sprint Planning y la Daily organizan el trabajo cercano del proyecto. No tiene sentido detallar por adelantado todos los Sprints de varios releases: se planifica con mayor precisión el Sprint que está por comenzar.

Tablero de trabajo

Un tablero básico visualiza **To Do → Doing → Done**. El equipo toma trabajo mediante un sistema *pull*: cada Product Developer se compromete con un ítem, lo lleva a Doing y lo mueve a Done al cumplir el criterio de terminado. **Tablero** no significa automáticamente **Kanban**: distintos enfoques usan tableros con reglas diferentes.

Cuando existen especialidades técnicas, el equipo puede descomponer cada User Story en **tareas** estimadas en horas ideales. El tablero permite seguir esas tareas, pero una tarea en Done no significa que toda la historia esté terminada. La historia llega a Done cuando se completó e integró el trabajo necesario y se cumple su De-

definition of Done. La responsabilidad por la historia y el incremento es del **equipo**, aunque las tareas individuales se autoasignen. Esta descomposición se trabaja en el «cómo» de la Planning; equipos maduros pueden necesitar menos detalle explícito.

6. Seguimiento visual: Burndown y Burnup

El **Sprint Burndown** muestra el *trabajo restante* durante un Sprint: en el eje horizontal se ubican los días y en el vertical los puntos de historia pendientes. La línea ideal baja desde el compromiso inicial hasta cero al cierre; la real se actualiza conforme se completan historias. Puede permanecer horizontal varios días y descender de golpe si las historias son grandes. Un aumento puede indicar trabajo descubierto o agregado y exige entender la causa, en lugar de interpretar el gráfico aisladamente.

El **Burnup** muestra *producto acumulado* a través de varios Sprints de un release o producto: puntos de historia entregados frente al tiempo. Se actualiza con los resultados aceptados al finalizar cada Sprint y permite comparar avance real y planificación. El Burndown pertenece a un Sprint concreto; el Burnup conserva la perspectiva de evolución a través de Sprints sucesivos.

Alcance del parcial: se aclaró que el tema formal de métricas se evalúa en el segundo parcial. Los gráficos se explicaron en el repaso para entender el seguimiento y la planificación, aunque no integran el temario de métricas de este primero.

7. Estimaciones tradicionales y ágiles

En una estimación **tradicional** se sigue una cadena de dependencias: **tamaño del producto** → **esfuerzo** → **duración/calendario** → **costo** → **presupuesto**. Primero se estima qué se va a construir usando una medida disponible en ese momento (por ejemplo, requerimientos, casos de uso o puntos de función; las líneas de código son difíciles de conocer al inicio). Del tamaño se deriva el esfuerzo, medido en **horas-persona**; de este se estima la duración. El costo se calcula después porque acortar el calendario puede cambiarlo mediante horas extra u otros recursos. A los costos del esfuerzo se suman infraestructura y demás gastos para preparar el presupuesto.

Un **recurso crítico** es aquel cuya disponibilidad es menor que la necesidad del proyecto; por ejemplo, licencias escasas, equipos especiales, sensores o una cuota limitada de tokens. Hace falta planificar su uso para que el equipo no quede esperando. Si cambia el alcance en un enfoque tradicional, se debe revisar el tamaño y las estimaciones derivadas, lo cual explica el costo de administrar cambios; no significa que los cambios estén prohibidos.

La estimación **ágil** es **relativa**: se comparan User Stories con una historia de referencia y se asignan **Story Points**. Un punto combina **complejidad, incertidumbre y esfuerzo** en una ponderación que permite comparar historias. Debe justificarse el valor considerando las tres dimensiones. Los puntos de un equipo no se trasladan mecánicamente a otro, porque la experiencia y las capacidades influyen en el esfuerzo. En el contraste planteado, la estimación tradicional busca mayor precisión numérica, mientras la ágil procura una previsión suficiente para decidir sin pagar el costo de una precisión falsa.

Planning Poker permite que quienes harán el trabajo contrasten su juicio con el conocimiento de otros integrantes y lleguen a una estimación compartida. Las diferencias de opinión ayudan a descubrir supuestos, sobreestimaciones y subestimaciones; la técnica no elimina la responsabilidad de estimar de quienes desarrollarán el producto.

Capacidad es una estimación previa del trabajo que el equipo puede asumir en el Sprint: se consideran días disponibles, integrantes y demás compromisos. Puede expresarse en horas ideales o puntos, según la madurez del equipo. **Velocidad** es una medición posterior: puntos de historias que el Product Owner aceptó al cierre. Es una referencia histórica, no una promesa de repetir automáticamente el mismo valor. La capacidad ayuda a decidir hasta dónde seleccionar historias del Product Backlog para el Sprint Backlog.

8. Gestión de configuración: preguntas posibles del parcial

La **gestión de configuración de software** sigue siendo necesaria con Scrum, con Kanban o con un proceso tradicional. Antes de organizar un repositorio hay que identificar qué elementos se controlarán, dónde estarán, cómo se nombrarán y versionarán, y cómo se gestionarán sus cambios. La estructura de repositorio puede ser objeto de una consigna de parcial.

Actividad	Qué permite hacer
Identificación	Elegir ítems de configuración y definir organización, nombres, versiones y repositorios.
Control de cambios	Establecer cómo se proponen, evalúan, autorizan e incorporan cambios.
Información de estado	Conocer y reportar el estado de la configuración: versiones, cambios y responsables de modificaciones.
Auditoría	Verificar de manera objetiva e independiente la configuración. La revisión entre integrantes del mismo equipo es una revisión técnica, no una auditoría independiente.

Las auditorías tradicionales pueden ser **físicas** y **funcionales**; se mencionó realizar primero la física y luego la funcional, con al menos una verificación antes de liberar una versión. El punto distintivo de una auditoría es la independencia de quienes la realizan, aun si pertenecen a la misma organización. La identificación, el control de cambios y la información de estado son necesarios cualquiera sea el enfoque de trabajo.